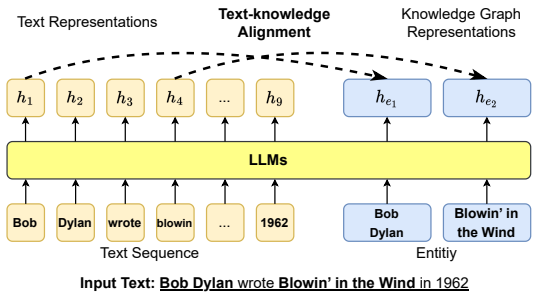

Fig. 9. Injecting KG information into LLMs training objective via text-knowledge alignment loss, where h denotes the hidden representation generated by LLMs.

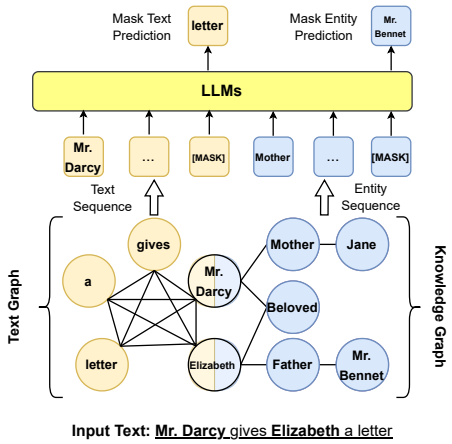

Fig. 10. Injecting KG information into LLMs inputs using graph structure.

sentiment words in the word masking objective.

The other line of work explicitly leverages the connections with knowledge and input text. As shown in Fig. 9, ERNIE [35] proposes a novel word-entity alignment training objective as a pre-training objective. Specifically, ERNIE feeds both sentences and corresponding entities mentioned in the text into LLMs, and then trains the LLMs to predict alignment links between textual tokens and entities in knowledge graphs. Similarly, KALM [91] enhances the input tokens by incorporating entity embeddings and includes an entity prediction pre-training task in addition to the token-only pre-training objective. This approach aims to improve the ability of LLMs to capture knowledge related to entities. Finally, KEPLER [40] directly employs both knowledge graph embedding training objective and Masked token pre-training objective into a shared transformer-based encoder. Deterministic LLM [104] focuses on pre-training language models to capture *deterministic* factual knowledge. It only masks the span that has a deterministic entity as the question and introduces additional clue contrast learning and clue classification objective. WKLM [106] first replaces entities in the text with other same-type entities and then feeds them into LLMs. The model is further pre-trained to distinguish whether the entities have been replaced or not.

4.1.2 Integrating KGs into LLM Inputs

As shown in Fig. 10, this kind of research focus on introducing relevant knowledge sub-graph into the inputs of LLMs. Given a knowledge graph triple and the corresponding sentences, ERNIE 3.0 [101] represents the triple as a sequence of tokens and directly concatenates them with the sentences. It further randomly masks either the relation token in the triple or tokens in the sentences to better combine knowledge with textual representations. However, such direct knowledge triple concatenation method allows the tokens in the sentence to intensively interact with the tokens in the knowledge sub-graph, which could result in *Knowledge Noise* [36]. To solve this issue, K-BERT [36] takes the first step to inject the knowledge triple into the sentence via a *visible matrix* where only the knowledge entities have access to the knowledge triple information, while the tokens in the sentences can only see each other in the self-attention module. To further reduce *Knowledge Noise*, Colake [107] proposes a unified word-knowledge graph (shown in Fig. 10) where the tokens in the input sentences form a fully connected word graph where tokens aligned with knowledge entities are connected with their neighboring entities.

The above methods can indeed inject a large amount of knowledge into LLMs. However, they mostly focus on popular entities and overlook the low-frequent and long-tail ones. DkLLM [108] aims to improve the LLMs representations towards those entities. DkLLM first proposes a novel measurement to determine long-tail entities and then replaces these selected entities in the text with pseudo token embedding as new input to the large language models. Furthermore, Dict-BERT [125] proposes to leverage external dictionaries to solve this issue. Specifically, Dict-BERT improves the representation quality of rare words by appending their definitions from the dictionary at the end of input text and trains the language model to locally align rare word representations in input sentences and dictionary definitions as well as to discriminate whether the input text and definition are correctly mapped.

4.1.3 KGs Instruction-tuning

Instead of injecting factual knowledge into LLMs, the KGs Instruction-tuning aims to fine-tune LLMs to better comprehend the structure of KGs and effectively follow user instructions to conduct complex tasks. KGs Instruction-tuning utilizes both facts and the structure of KGs to create instruction-tuning datasets. LLMs finetuned on these datasets can extract both factual and structural knowledge from KGs, enhancing the reasoning ability of LLMs. KP-PLM [109] first designs several prompt templates to transfer structural graphs into natural language text. Then, two self-supervised tasks are proposed to finetune LLMs to further leverage the knowledge from these prompts. OntoPrompt [110] proposes an ontology-enhanced prompt-tuning that can place knowledge of entities into the context of LLMs, which are further finetuned on several downstream tasks. ChatKBQA [111] finetunes LLMs on KG structure to generate logical queries, which can be executed on KGs to obtain answers. To better reason on graphs, RoG [112] presents a planning-retrieval-reasoning framework. RoG is finetuned on KG structure to generate relation paths grounded by KGs as faithful plans. These plans are then used to retrieve valid

reasoning paths from the KGs for LLMs to conduct faithful reasoning and generate interpretable results.

KGs Instruction-tuning can better leverage the knowledge from KGs for downstream tasks. However, it requires retraining the models, which is time-consuming and requires lots of resources.

4.2 KG-enhanced LLM Inference

The above methods could effectively fuse knowledge into LLMs. However, real-world knowledge is subject to change and the limitation of these approaches is that they do not permit updates to the incorporated knowledge without retraining the model. As a result, they may not generalize well to the unseen knowledge during inference [126]. Therefore, considerable research has been devoted to keeping the knowledge space and text space separate and injecting the knowledge while inference. These methods mostly focus on the Question Answering (QA) tasks, because QA requires the model to capture both textual semantic meanings and up-to-date real-world knowledge.

4.2.1 Retrieval-Augmented Knowledge Fusion

Retrieval-Augmented Knowledge Fusion is a popular method to inject knowledge into LLMs during inference. The key idea is to retrieve relevant knowledge from a large corpus and then fuse the retrieved knowledge into LLMs. As shown in Fig. 11, RAG [92] proposes to combine non-parametric and parametric modules to handle the external knowledge. Given the input text, RAG first searches for relevant KG in the non-parametric module via MIPS to obtain several documents. RAG then treats these documents as hidden variables z and feeds them into the output generator, empowered by Seq2Seq LLMs, as additional context information. The research indicates that using different retrieved documents as conditions at different generation steps performs better than only using a single document to guide the whole generation process. The experimental results show that RAG outperforms other parametric-only and non-parametric-only baseline models in open-domain QA. RAG can also generate more specific, diverse, and factual text than other parameter-only baselines. Story-fragments [127] further improves architecture by adding an additional module to determine salient knowledge entities and fuse them into the generator to improve the quality of generated long stories. EMAT [115] further improves the efficiency of such a system by encoding external knowledge into a key-value memory and exploiting the fast maximum inner product search for memory querying. REALM [114] proposes a novel knowledge retriever to help the model to retrieve and attend over documents from a large corpus during the pre-training stage and successfully improves the performance of open-domain question answering. KGLM [113] selects the facts from a knowledge graph using the current context to generate factual sentences. With the help of an external knowledge graph, KGLM could describe facts using out-of-domain words or phrases.

4.2.2 KGs Prompting

To better feed the KG structure into the LLM during inference, KGs prompting aims to design a crafted prompt that

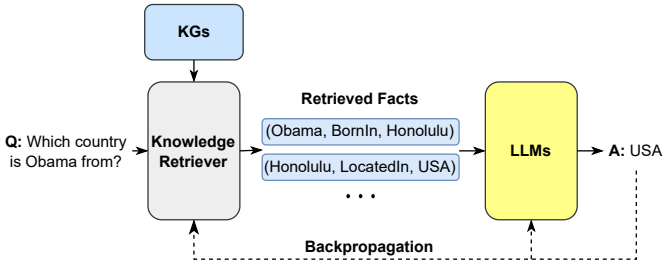

Fig. 11. Retrieving external knowledge to enhance the LLM generation.

converts structured KGs into text sequences, which can be fed as context into LLMs. In this way, LLMs can better take advantage of the structure of KGs to perform reasoning. Li et al. [64] adopt the pre-defined template to convert each triple into a short sentence, which can be understood by LLMs for reasoning. Mindmap [65] designs a KG prompt to convert graph structure into a mind map that enables LLMs to perform reasoning by consolidating the facts in KGs and the implicit knowledge from LLMs. ChatRule [116] samples several relation paths from KGs, which are verbalized and fed into LLMs. Then, LLMs are prompted to generate meaningful logical rules that can be used for reasoning. CoK [117] proposes a chain-of-knowledge prompting that uses a sequence of triples to elicit the reasoning ability of LLMs to reach the final answer.

KGs prompting presents a simple way to synergize LLMs and KGs. By using the prompt, we can easily harness the power of LLMs to perform reasoning based on KGs without retraining the models. However, the prompt is usually designed manually, which requires lots of human effort.

4.3 Comparison between KG-enhanced LLM Pre-training and Inference

KG-enhanced LLM Pre-training methods commonly enrich large-amount of unlabeled corpus with semantically relevant real-world knowledge. These methods allow the knowledge representations to be aligned with appropriate linguistic context and explicitly train LLMs to leverage those knowledge from scratch. When applying the resulting LLMs to downstream knowledge-intensive tasks, they should achieve optimal performance. In contrast, KG-enhanced LLM inference methods only present the knowledge to LLMs in the inference stage and the underlying LLMs may not be trained to fully leverage these knowledge when conducting downstream tasks, potentially resulting in sub-optimal model performance.

However, real-world knowledge is dynamic and requires frequent updates. Despite being effective, the KG-enhanced LLM Pre-training methods never permit knowledge updates or editing without model re-training. As a result, the KG-enhanced LLM Pre-training methods could generalize poorly to recent or unseen knowledge. KG-enhanced LLM inference methods can easily maintain knowledge updates by changing the inference inputs. These methods help improve LLMs performance on new knowledge and domains.

In summary, when to use these methods depends on the application scenarios. If one wishes to apply LLMs to han-

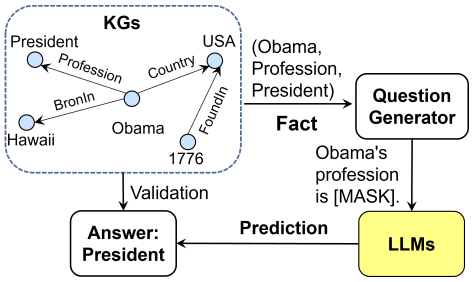

Fig. 12. The general framework of using knowledge graph for language model probing.

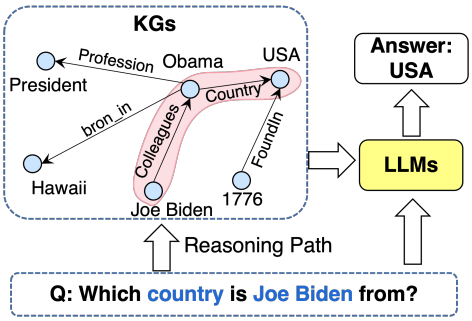

Fig. 13. The general framework of using knowledge graph for language model analysis.

dle time-insensitive knowledge in particular domains (e.g., commonsense and reasoning knowledge), KG-enhanced LLM Pre-training methods should be considered. Otherwise, KG-enhanced LLM inference methods can be used to handle open-domain knowledge with frequent updates.

4.4 KG-enhanced LLM Interpretability

Although LLMs have achieved remarkable success in many NLP tasks, they are still criticized for their lack of interpretability. The large language model (LLM) interpretability refers to the understanding and explanation of the inner workings and decision-making processes of a large language model [17]. This can improve the trustworthiness of LLMs and facilitate their applications in high-stakes scenarios such as medical diagnosis and legal judgment. Knowledge graphs (KGs) represent the knowledge structurally and can provide good interpretability for the reasoning results. Therefore, researchers try to utilize KGs to improve the interpretability of LLMs, which can be roughly grouped into two categories: *1) KGs for language model probing*, and *2) KGs for language model analysis*.

4.4.1 KGs for LLM Probing

The large language model (LLM) probing aims to understand the knowledge stored in LLMs. LLMs, trained on large-scale corpus, are often known as containing enormous knowledge. However, LLMs store the knowledge in a hidden way, making it hard to figure out the stored knowledge. Moreover, LLMs suffer from the hallucination problem [15], which results in generating statements that contradict facts. This issue significantly affects the reliability of LLMs. Therefore, it is necessary to probe and verify the knowledge stored in LLMs.

LAMA [14] is the first work to probe the knowledge in LLMs by using KGs. As shown in Fig. 12, LAMA first converts the facts in KGs into cloze statements by a predefined prompt template and then uses LLMs to predict the missing entity. The prediction results are used to evaluate the knowledge stored in LLMs. For example, we try to probe whether LLMs know the fact *(Obama, profession, president)*. We first convert the fact triple into a cloze question "Obama's profession is _." with the object masked. Then, we test if the LLMs can predict the object "president" correctly.

However, LAMA ignores the fact that the prompts are inappropriate. For example, the prompt *"Obama worked as a _"* may be more favorable to the prediction of the blank by the language models than *"Obama is a _ by profession"*. Thus, LPAQA [118] proposes a mining and paraphrasing-based method to automatically generate high-quality and diverse prompts for a more accurate assessment of the knowledge contained in the language model. Moreover, Adolphs et al. [128] attempt to use examples to make the language model understand the query, and experiments obtain substantial improvements for BERT-large on the T-REx data. Unlike using manually defined prompt templates, Autoprompt [119] proposes an automated method, which is based on the gradient-guided search to create prompts. LLM-facteval [121] designs a systematic framework that automatically generates probing questions from KGs. The generated questions are then used to evaluate the factual knowledge stored in LLMs.

Instead of probing the general knowledge by using the encyclopedic and commonsense knowledge graphs, BioLAMA [129] and MedLAMA [120] probe the medical knowledge in LLMs by using medical knowledge graphs. Alex et al. [130] investigate the capacity of LLMs to retain less popular factual knowledge. They select unpopular facts from Wikidata knowledge graphs which have low-frequency clicked entities. These facts are then used for the evaluation, where the results indicate that LLMs encounter difficulties with such knowledge, and that scaling fails to appreciably improve memorization of factual knowledge in the tail.

4.4.2 KGs for LLM Analysis

Knowledge graphs (KGs) for pre-train language models (LLMs) analysis aims to answer the following questions such as "how do LLMs generate the results?", and "how do the function and structure work in LLMs?". To analyze the inference process of LLMs, as shown in Fig. 13, KagNet [38] and QA-GNN [131] make the results generated by LLMs at each reasoning step grounded by knowledge graphs. In this way, the reasoning process of LLMs can be explained by extracting the graph structure from KGs. Shaobo et al. [123] investigate how LLMs generate the results correctly. They adopt the causal-inspired analysis from facts extracted from KGs. This analysis quantitatively measures the word patterns that LLMs depend on to generate the results. The results show that LLMs generate the missing factual more by the positionally closed words rather than the knowledge-dependent words. Thus, they claim that LLMs are inadequate to memorize factual knowledge because of the inaccurate dependence. To interpret the training of LLMs, Swamy

TABLE 3
Summary of representative LLM-augmented KG methods.

Task	Method	Year	LLM	Technique
LLM-augmented KG embedding	Pretrain-KGE [94]	2020	**E**	LLMs as Text Encoders
	KEPLER [40]	2020	**E**	LLMs as Text Encoders
	Nayyeri et al. [132]	2022	**E**	LLMs as Text Encoders
	Huang et al. [133]	2022	**E**	LLMs as Text Encoders
	CoDEx [134]	2022	**E**	LLMs as Text Encoders
	LMKE [135]	2022	**E**	LLMs for Joint Text and KG Embedding
	kNN-KGE [136]	2022	**E**	LLMs for Joint Text and KG Embedding
	LambdaKG [137]	2023	**E** + **D** + **ED**	LLMs for Joint Text and KG Embedding
LLM-augmented KG completion	KG-BERT [26]	2019	**E**	Joint Encoding
	MTL-KGC [138]	2020	**E**	Joint Encoding
	PKGC [139]	2022	**E**	Joint Encoding
	LASS [140]	2022	**E**	Joint Encoding
	MEM-KGC [141]	2021	**E**	MLM Encoding
	OpenWorld KGC [142]	2023	**E**	MLM Encoding
	StAR [143]	2021	**E**	Separated Encoding
	SimKGC [144]	2022	**E**	Separated Encoding
	LP-BERT [145]	2022	**E**	Separated Encoding
	GenKGC [96]	2022	**ED**	LLM as decoders
	KGT5 [146]	2022	**ED**	LLM as decoders
	KG-S2S [147]	2022	**ED**	LLM as decoders
	AutoKG [93]	2023	**D**	LLM as decoders
LLM-augmented KG construction	ELMO [148]	2018	**E**	Named Entity Recognition
	GenerativeNER [149]	2021	**ED**	Named Entity Recognition
	LDET [150]	2019	**E**	Entity Typing
	BOX4Types [151]	2021	**E**	Entity Typing
	ELQ [152]	2020	**E**	Entity Linking
	ReFinED [153]	2022	**E**	Entity Linking
	BertCR [154]	2019	**E**	CR (Within-document)
	Spanbert [155]	2020	**E**	CR (Within-document)
	CDLM [156]	2021	**E**	CR (Cross-document)
	CrossCR [157]	2021	**E**	CR (Cross-document)
	CR-RL [158]	2021	**E**	CR (Cross-document)
	SentRE [159]	2019	**E**	RE (Sentence-level)
	Curriculum-RE [160]	2021	**E**	RE (Sentence-level)
	DREEAM [161]	2023	**E**	RE (Document-level)
	Kumar et al. [95]	2020	**E**	End-to-End Construction
	Guo et al. [162]	2021	**E**	End-to-End Construction
	Grapher [41]	2021	**ED**	End-to-End Construction
	PiVE [163]	2023	**D** + **ED**	End-to-End Construction
	COMET [164]	2019	**D**	Distilling KGs from LLMs
	BertNet [165]	2022	**E**	Distilling KGs from LLMs
	West et al. [166]	2022	**D**	Distilling KGs from LLMs
LLM-augmented KG-to-text Generation	Ribeiro et al [167]	2021	**ED**	Leveraging Knowledge from LLMs
	JointGT [42]	2021	**ED**	Leveraging Knowledge from LLMs
	FSKG2Text [168]	2021	**D** + **ED**	Leveraging Knowledge from LLMs
	GAP [169]	2022	**ED**	Leveraging Knowledge from LLMs
	GenWiki [170]	2020	-	Constructing KG-text aligned Corpus
	KGPT [171]	2020	**ED**	Constructing KG-text aligned Corpus
LLM-augmented KGQA	Lukovnikov et al. [172]	2019	**E**	Entity/Relation Extractor
	Luo et al. [173]	2020	**E**	Entity/Relation Extractor
	QA-GNN [131]	2021	**E**	Entity/Relation Extractor
	Nan et al. [174]	2023	**E** + **D** + **ED**	Entity/Relation Extractor
	DEKCOR [175]	2021	**E**	Answer Reasoner
	DRLK [176]	2022	**E**	Answer Reasoner
	OreoLM [177]	2022	**E**	Answer Reasoner
	GreaseLM [178]	2022	**E**	Answer Reasoner
	ReLMKG [179]	2022	**E**	Answer Reasoner
	UniKGQA [43]	2023	**E**	Answer Reasoner

E: Encoder-only LLMs, **D**: Decoder-only LLMs, **ED**: Encoder-decoder LLMs.

et al. [122] adopt the language model during pre-training to generate knowledge graphs. The knowledge acquired by LLMs during training can be unveiled by the facts in KGs explicitly. To explore how implicit knowledge is stored in parameters of LLMs, Dai et al. [39] propose the concept of *knowledge neurons*. Specifically, activation of the identified knowledge neurons is highly correlated with knowledge expression. Thus, they explore the knowledge and facts represented by each neuron by suppressing and amplifying knowledge neurons.

5 LLM-AUGMENTED KGS

Knowledge graphs are famous for representing knowledge in a structural manner. They have been applied in many downstream tasks such as question answering, recommendation, and web search. However, the conventional KGs are often incomplete and existing methods often lack considering textual information. To address these issues, recent research has explored integrating LLMs to augment KGs to consider the textual information and improve the performance in downstream tasks. In this section, we will introduce the recent research on LLM-augmented KGs. We will introduce the methods that integrate LLMs for KG embedding, KG completion, KG construction, KG-to-text generation, and KG question answering, respectively. Representative works are summarized in Table 3.

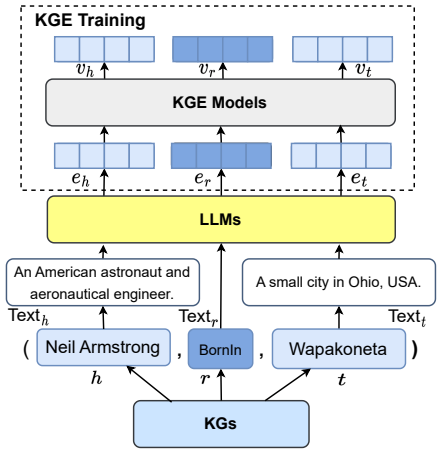

Fig. 14. LLMs as text encoder for knowledge graph embedding (KGE).

5.1 LLM-augmented KG Embedding

Knowledge graph embedding (KGE) aims to map each entity and relation into a low-dimensional vector (embedding) space. These embeddings contain both semantic and structural information of KGs, which can be utilized for various tasks such as question answering [180], reasoning [38], and recommendation [181]. Conventional knowledge graph embedding methods mainly rely on the structural information of KGs to optimize a scoring function defined on embeddings (e.g., TransE [33], and DisMult [182]). However, these approaches often fall short in representing unseen entities and long-tailed relations due to their limited structural connectivity [183], [184]. To address this issue, as shown in Fig. 14, recent research adopts LLMs to enrich representations of KGs by encoding the textual descriptions of entities and relations [40], [94].

5.1.1 LLMs as Text Encoders

Pretrain-KGE [94] is a representative method that follows the framework shown in Fig. 14. Given a triple (h, r, t) from KGs, it firsts uses a LLM encoder to encode the textual descriptions of entities h, t, and relations r into representations as

$$e_h = \text{LLM}(\text{Text}_h), e_t = \text{LLM}(\text{Text}_t), e_r = \text{LLM}(\text{Text}_r), \quad (1)$$

where e_h, e_r, and e_t denotes the initial embeddings of entities h, t, and relations r, respectively. Pretrain-KGE uses the BERT as the LLM encoder in experiments. Then, the initial embeddings are fed into a KGE model to generate the final embeddings v_h, v_r, and v_t. During the KGE training phase, they optimize the KGE model by following the standard KGE loss function as

$$\mathcal{L} = [\gamma + f(v_h, v_r, v_t) - f(v'_h, v'_r, v'_t)], \quad (2)$$

where f is the KGE scoring function, γ is a margin hyperparameter, and v'_h, v'_r, and v'_t are the negative samples. In this way, the KGE model could learn adequate structure information, while reserving partial knowledge from LLM enabling better knowledge graph embedding. KEPLER [40] offers a unified model for knowledge embedding and pre-trained language representation. This model not only generates effective text-enhanced knowledge embedding

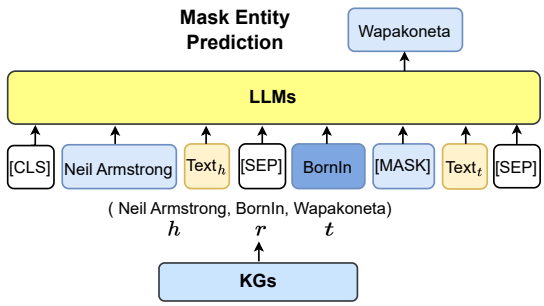

Fig. 15. LLMs for joint text and knowledge graph embedding.

using powerful LLMs but also seamlessly integrates factual knowledge into LLMs. Nayyeri et al. [132] use LLMs to generate the world-level, sentence-level, and document-level representations. They are integrated with graph structure embeddings into a unified vector by Dihedron and Quaternion representations of 4D hypercomplex numbers. Huang et al. [133] combine LLMs with other vision and graph encoders to learn multi-modal knowledge graph embedding that enhances the performance of downstream tasks. CoDEx [134] presents a novel loss function empowered by LLMs that guides the KGE models in measuring the likelihood of triples by considering the textual information. The proposed loss function is agnostic to model structure that can be incorporated with any KGE model.

5.1.2 LLMs for Joint Text and KG Embedding

Instead of using KGE model to consider graph structure, another line of methods directly employs LLMs to incorporate both the graph structure and textual information into the embedding space simultaneously. As shown in Fig. 15, kNN-KGE [136] treats the entities and relations as special tokens in the LLM. During training, it transfers each triple (h, r, t) and corresponding text descriptions into a sentence x as

$$x = \texttt{[CLS]}\ h\ \text{Text}_h\ \texttt{[SEP]}\ r\ \texttt{[SEP]}\ \texttt{[MASK]}\ \text{Text}_t\ \texttt{[SEP]}, \tag{3}$$

where the tailed entities are replaced by `[MASK]`. The sentence is fed into a LLM, which then finetunes the model to predict the masked entity, formulated as

$$P_{LLM}(t|h, r) = P(\texttt{[MASK]=t}|x, \Theta), \tag{4}$$

where Θ denotes the parameters of the LLM. The LLM is optimized to maximize the probability of the correct entity t. After training, the corresponding token representations in LLMs are used as embeddings for entities and relations. Similarly, LMKE [135] proposes a contrastive learning method to improve the learning of embeddings generated by LLMs for KGE. Meanwhile, to better capture graph structure, LambdaKG [137] samples 1-hop neighbor entities and concatenates their tokens with the triple as a sentence feeding into LLMs.

5.2 LLM-augmented KG Completion

Knowledge Graph Completion (KGC) refers to the task of inferring missing facts in a given knowledge graph. Similar to KGE, conventional KGC methods mainly focused on the structure of the KG, without considering the extensive textual information. However, the recent integration of LLMs enables KGC methods to encode text or generate facts for better KGC performance. These methods fall into two distinct categories based on their utilization styles: *1) LLM as Encoders (PaE)*, and *2) LLM as Generators (PaG)*.

5.2.1 LLM as Encoders (PaE).

As shown in Fig. 16 (a), (b), and (c), this line of work first uses encoder-only LLMs to encode textual information as well as KG facts. Then, they predict the plausibility of the triples or masked entities by feeding the encoded representation into a prediction head, which could be a simple MLP or conventional KG score function (e.g., TransE [33] and TransR [185]).

Joint Encoding. Since the encoder-only LLMs (e.g., Bert [1]) are well at encoding text sequences, KG-BERT [26] represents a triple (h, r, t) as a text sequence and encodes it with LLM Fig. 16(a).

$$x = \texttt{[CLS]}\ \text{Text}_h\ \texttt{[SEP]}\ \text{Text}_r\ \texttt{[SEP]}\ \text{Text}_t\ \texttt{[SEP]}, \tag{5}$$

The final hidden state of the `[CLS]` token is fed into a classifier to predict the possibility of the triple, formulated as

$$s = \sigma(\text{MLP}(e_{\texttt{[CLS]}})), \tag{6}$$

where $\sigma(\cdot)$ denotes the sigmoid function and $e_{\texttt{[CLS]}}$ denotes the representation encoded by LLMs. To improve the efficacy of KG-BERT, MTL-KGC [138] proposed a Multi-Task Learning for the KGC framework which incorporates additional auxiliary tasks into the model's training, i.e. prediction (RP) and relevance ranking (RR). PKGC [139] assesses the validity of a triplet (h, r, t) by transforming the triple and its supporting information into natural language sentences with pre-defined templates. These sentences are then processed by LLMs for binary classification. The supporting information of the triplet is derived from the attributes of h and t with a verbalizing function. For instance, if the triple is *(Lebron James, member of sports team, Lakers)*, the information regarding Lebron James is verbalized as "Lebron James: American basketball player". LASS [140] observes that language semantics and graph structures are equally vital to KGC. As a result, LASS is proposed to jointly learn two types of embeddings: semantic embedding and structure embedding. In this method, the full text of a triple is forwarded to the LLM, and the mean pooling of the corresponding LLM outputs for h, r, and t are separately calculated. These embeddings are then passed to a graph-based method, i.e. TransE, to reconstruct the KG structures.

MLM Encoding. Instead of encoding the full text of a triple, many works introduce the concept of Masked Language Model (MLM) to encode KG text (Fig. 16(b)). MEM-KGC [141] uses Masked Entity Model (MEM) classification mechanism to predict the masked entities of the triple. The input text is in the form of

$$x = \texttt{[CLS]}\ \text{Text}_h\ \texttt{[SEP]}\ \text{Text}_r\ \texttt{[SEP]}\ \texttt{[MASK]}\ \texttt{[SEP]}, \tag{7}$$

Similar to Eq. 4, it tries to maximize the probability that the masked entity is the correct entity t. Additionally, to enable the model to learn unseen entities, MEM-KGC integrates

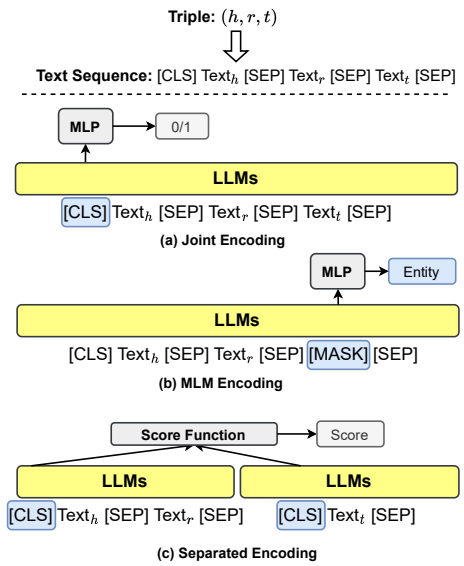

Fig. 16. The general framework of adopting LLMs as encoders (PaE) for KG Completion.

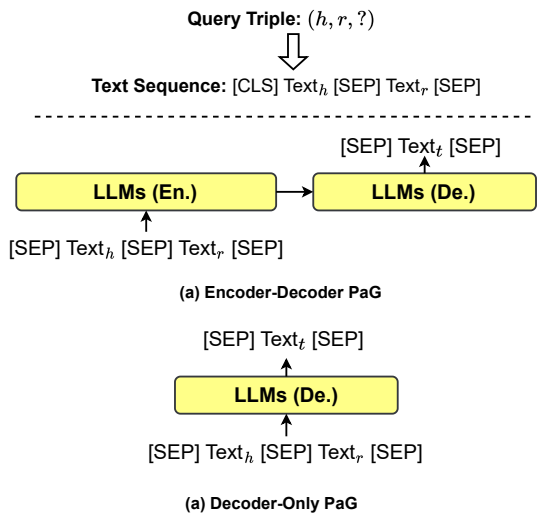

Fig. 17. The general framework of adopting LLMs as decoders (PaG) for KG Completion. The En. and De. denote the encoder and decoder, respectively.

multitask learning for entities and super-class prediction based on the text description of entities:

$$x = \texttt{[CLS] [MASK] [SEP]}\ \text{Text}_h\ \texttt{[SEP]}. \tag{8}$$

OpenWorld KGC [142] expands the MEM-KGC model to address the challenges of open-world KGC with a pipeline framework, where two sequential MLM-based modules are defined: Entity Description Prediction (EDP), an auxiliary module that predicts a corresponding entity with a given textual description; Incomplete Triple Prediction (ITP), the target module that predicts a plausible entity for a given incomplete triple $(h, r, ?)$. EDP first encodes the triple with Eq. 8 and generates the final hidden state, which is then forwarded into ITP as an embedding of the head entity in Eq. 7 to predict target entities.

Separated Encoding. As shown in Fig. 16(c), these methods involve partitioning a triple (h, r, t) into two distinct parts, i.e. (h, r) and t, which can be expressed as

$$x_{(h,r)} = \texttt{[CLS]}\ \text{Text}_h\ \texttt{[SEP]}\ \text{Text}_r\ \texttt{[SEP]}, \tag{9}$$

$$x_t = \texttt{[CLS]}\ \text{Text}_t\ \texttt{[SEP]}. \tag{10}$$

Then the two parts are encoded separately by LLMs, and the final hidden states of the [CLS] tokens are used as the representations of (h, r) and t, respectively. The representations are then fed into a scoring function to predict the possibility of the triple, formulated as

$$s = f_{score}(e_{(h,r)}, e_t), \tag{11}$$

where f_{score} denotes the score function like TransE.

StAR [143] applies Siamese-style textual encoders on their text, encoding them into separate contextualized representations. To avoid the combinatorial explosion of textual encoding approaches, e.g., KG-BERT, StAR employs a scoring module that involves both deterministic classifier and spatial measurement for representation and structure learning respectively, which also enhances structured knowledge by exploring the spatial characteristics. SimKGC [144] is another instance of leveraging a Siamese textual encoder to encode textual representations. Following the encoding process, SimKGC applies contrastive learning techniques to these representations. This process involves computing the similarity between the encoded representations of a given triple and its positive and negative samples. In particular, the similarity between the encoded representation of the triple and the positive sample is maximized, while the similarity between the encoded representation of the triple and the negative sample is minimized. This enables SimKGC to learn a representation space that separates plausible and implausible triples. To avoid overfitting textural information, CSPromp-KG [186] employs parameter-efficient prompt learning for KGC.

LP-BERT [145] is a hybrid KGC method that combines both MLM Encoding and Separated Encoding. This approach consists of two stages, namely pre-training and fine-tuning. During pre-training, the method utilizes the standard MLM mechanism to pre-train a LLM with KGC data. During the fine-tuning stage, the LLM encodes both parts and is optimized using a contrastive learning strategy (similar to SimKGC [144]).

5.2.2 *LLM as Generators (PaG).*

Recent works use LLMs as sequence-to-sequence generators in KGC. As presented in Fig. 17 (a) and (b), these approaches involve encoder-decoder or decoder-only LLMs. The LLMs receive a sequence text input of the query triple $(h, r, ?)$, and generate the text of tail entity t directly.

GenKGC [96] uses the large language model BART [5] as the backbone model. Inspired by the in-context learning approach used in GPT-3 [59], where the model concatenates relevant samples to learn correct output answers, GenKGC proposes a relation-guided demonstration technique that includes triples with the same relation to facilitating the model's learning process. In addition, during generation, an entity-aware hierarchical decoding method is proposed to reduce the time complexity. KGT5 [146] introduces a

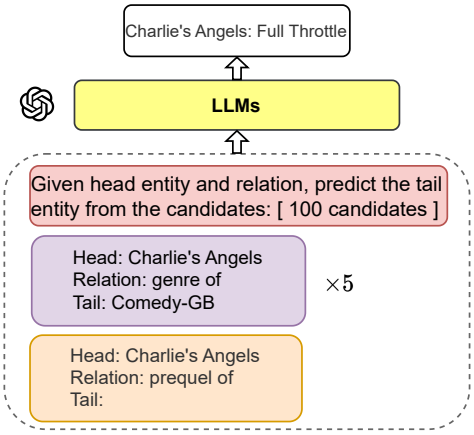

Fig. 18. The framework of prompt-based PaG for KG Completion.

novel KGC model that fulfils four key requirements of such models: scalability, quality, versatility, and simplicity. To address these objectives, the proposed model employs a straightforward T5 small architecture. The model is distinct from previous KGC methods, in which it is randomly initialized rather than using pre-trained models. KG-S2S [147] is a comprehensive framework that can be applied to various types of KGC tasks, including Static KGC, Temporal KGC, and Few-shot KGC. To achieve this objective, KG-S2S reformulates the standard triple KG fact by introducing an additional element, forming a quadruple (h, r, t, m), where m represents the additional "condition" element. Although different KGC tasks may refer to different conditions, they typically have a similar textual format, which enables unification across different KGC tasks. The KG-S2S approach incorporates various techniques such as entity description, soft prompt, and Seq2Seq Dropout to improve the model's performance. In addition, it utilizes constrained decoding to ensure the generated entities are valid. For closed-source LLMs (e.g., ChatGPT and GPT-4), AutoKG adopts prompt engineering to design customized prompts [93]. As shown in Fig. 18, these prompts contain the task description, few-shot examples, and test input, which instruct LLMs to predict the tail entity for KG completion.

Comparison between PaE and PaG. LLMs as Encoders (PaE) applies an additional prediction head on the top of the representation encoded by LLMs. Therefore, the PaE framework is much easier to finetune since we can only optimize the prediction heads and freeze the LLMs. Moreover, the output of the prediction can be easily specified and integrated with existing KGC functions for different KGC tasks. However, during the inference stage, the PaE requires to compute a score for every candidate in KGs, which could be computationally expensive. Besides, they cannot generalize to unseen entities. Furthermore, the PaE requires the representation output of the LLMs, whereas some state-of-the-art LLMs (e.g. GPT-4[1]) are closed sources and do not grant access to the representation output.

LLMs as Generators (PaG), on the other hand, which does not need the prediction head, can be used without finetuning or access to representations. Therefore, the framework of PaG is suitable for all kinds of LLMs. In addition, PaG directly generates the tail entity, making it efficient in inference without ranking all the candidates and easily generalizing to unseen entities. But, the challenge of PaG is that the generated entities could be diverse and not lie in KGs. What is more, the time of a single inference is longer due to the auto-regressive generation. Last, how to design a powerful prompt that feeds KGs into LLMs is still an open question. Consequently, while PaG has demonstrated promising results for KGC tasks, the trade-off between model complexity and computational efficiency must be carefully considered when selecting an appropriate LLM-based KGC framework.

5.2.3 Model Analysis

Justin et al. [187] provide a comprehensive analysis of KGC methods integrated with LLMs. Their research investigates the quality of LLM embeddings and finds that they are suboptimal for effective entity ranking. In response, they propose several techniques for processing embeddings to improve their suitability for candidate retrieval. The study also compares different model selection dimensions, such as Embedding Extraction, Query Entity Extraction, and Language Model Selection. Lastly, the authors propose a framework that effectively adapts LLM for knowledge graph completion.

5.3 LLM-augmented KG Construction

Knowledge graph construction involves creating a structured representation of knowledge within a specific domain. This includes identifying entities and their relationships with each other. The process of knowledge graph construction typically involves multiple stages, including *1) entity discovery*, *2) coreference resolution*, and *3) relation extraction*. Fig 19 presents the general framework of applying LLMs for each stage in KG construction. More recent approaches have explored *4) end-to-end knowledge graph construction*, which involves constructing a complete knowledge graph in one step or directly *5) distilling knowledge graphs from LLMs*.

5.3.1 Entity Discovery

Entity discovery in KG construction refers to the process of identifying and extracting entities from unstructured data sources, such as text documents, web pages, or social media posts, and incorporating them to construct knowledge graphs.

Named Entity Recognition (NER) involves identifying and tagging named entities in text data with their positions and classifications. The named entities include people, organizations, locations, and other types of entities. The state-of-the-art NER methods usually employ LLMs to leverage their contextual understanding and linguistic knowledge for accurate entity recognition and classification. There are three NER sub-tasks based on the types of NER spans identified, i.e., flat NER, nested NER, and discontinuous NER. *1) Flat NER is to identify non-overlapping named entities from input text.* It is usually conceptualized as a sequence labelling problem where each token in the text is assigned a unique label based on its position in the sequence [1], [148], [188], [189]. *2) Nested NER considers complex scenarios which allow a token to belong to multiple entities.* The span-based method [190]–[194] is a popular branch of nested

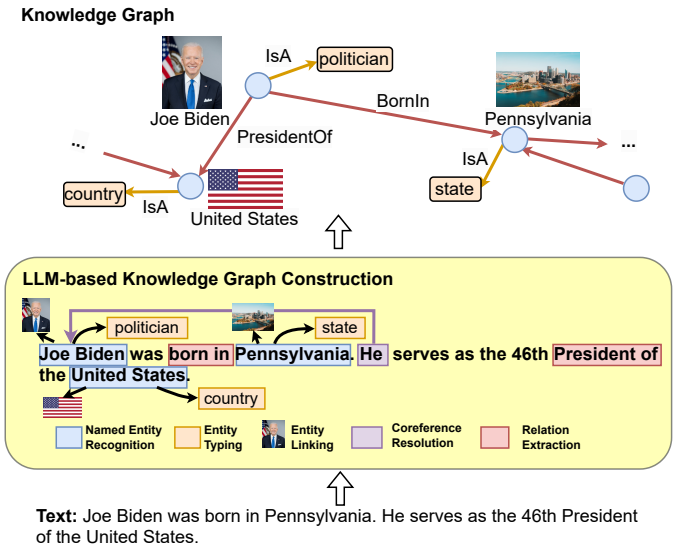

Fig. 19. The general framework of LLM-based KG construction.

NER which involves enumerating all candidate spans and classifying them into entity types (including a non-entity type). Parsing-based methods [195]–[197] reveal similarities between nested NER and constituency parsing tasks (predicting nested and non-overlapping spans), and propose to integrate the insights of constituency parsing into nested NER. *3) Discontinuous NER identifies named entities that may not be contiguous in the text.* To address this challenge, [198] uses the LLM output to identify entity fragments and determine whether they are overlapped or in succession.

Unlike the task-specific methods, GenerativeNER [149] uses a sequence-to-sequence LLM with a pointer mechanism to generate an entity sequence, which is capable of solving all three types of NER sub-tasks.

Entity Typing (ET) aims to provide fine-grained and ultra-grained type information for a given entity mentioned in context. These methods usually utilize LLM to encode mentions, context and types. LDET [150] applies pre-trained ELMo embeddings [148] for word representation and adopts LSTM as its sentence and mention encoders. BOX4Types [151] recognizes the importance of type dependency and uses BERT to represent the hidden vector and each type in a hyperrectangular (box) space. LRN [199] considers extrinsic and intrinsic dependencies between labels. It encodes the context and entity with BERT and employs these output embeddings to conduct deductive and inductive reasoning. MLMET [200] uses predefined patterns to construct input samples for the BERT MLM and employs [MASK] to predict context-dependent hypernyms of the mention, which can be viewed as type labels. PL [201] and DFET [202] utilize prompt learning for entity typing. LITE [203] formulates entity typing as textual inference and uses RoBERTa-large-MNLI as the backbone network.

Entity Linking (EL), as known as entity disambiguation, involves linking entity mentions appearing in the text to their corresponding entities in a knowledge graph. [204] proposed BERT-based end-to-end EL systems that jointly discover and link entities. ELQ [152] employs a fast bi-encoder architecture to jointly perform mention detection and linking in one pass for downstream question answering systems. Unlike previous models that frame EL as matching in vector space, GENRE [205] formulates it as a sequence-to-sequence problem, autoregressively generating a version of the input markup-annotated with the unique identifiers of an entity expressed in natural language. GENRE is extended to its multilingual version mGENRE [206]. Considering the efficiency challenges of generative EL approaches, [207] parallelizes autoregressive linking across all potential mentions and relies on a shallow and efficient decoder. ReFinED [153] proposes an efficient zero-shot-capable EL approach by taking advantage of fine-grained entity types and entity descriptions which are processed by a LLM-based encoder.

5.3.2 Coreference Resolution (CR)

Coreference resolution is to find all expressions (i.e., mentions) that refer to the same entity or event in a text.

Within-document CR refers to the CR sub-task where all these mentions are in a single document. Mandar et al. [154] initialize LLM-based coreferences resolution by replacing the previous LSTM encoder [208] with BERT. This work is followed by the introduction of SpanBERT [155] which is pre-trained on BERT architecture with a span-based masked language model (MLM). Inspired by these works, Tuan Manh et al. [209] present a strong baseline by incorporating the SpanBERT encoder into a non-LLM approach e2e-coref [208]. CorefBERT leverages Mention Reference Prediction (MRP) task which masks one or several mentions and requires the model to predict the masked mention's corresponding referents. CorefQA [210] formulates coreference resolution as a question answering task, where contextual queries are generated for each candidate mention and the coreferent spans are extracted from the document using the queries. Tuan Manh et al. [211] introduce a gating mechanism and a noisy training method to extract information from event mentions using the SpanBERT encoder.

In order to reduce the large memory footprint faced by large LLM-based NER models, Yuval et al. [212] and Raghuveer el al. [213] proposed start-to-end and approximation models, respectively, both utilizing bilinear functions to calculate mention and antecedent scores with reduced reliance on span-level representations.

Cross-document CR refers to the sub-task where the mentions refer to the same entity or event might be across multiple documents. CDML [156] proposes a cross document language modeling method which pre-trains a Longformer [214] encoder on concatenated related documents and employs an MLP for binary classification to determine whether a pair of mentions is coreferent or not. CrossCR [157] utilizes an end-to-end model for cross-document coreference resolution which pre-trained the mention scorer on gold mention spans and uses a pairwise scorer to compare mentions with all spans across all documents. CR-RL [158] proposes an actor-critic deep reinforcement learning-based coreference resolver for cross-document CR.

5.3.3 Relation Extraction (RE)

Relation extraction involves identifying semantic relationships between entities mentioned in natural language text. There are two types of relation extraction methods, i.e.

sentence-level RE and document-level RE, according to the scope of the text analyzed.

Sentence-level RE focuses on identifying relations between entities within a single sentence. Peng et al. [159] and TRE [215] introduce LLM to improve the performance of relation extraction models. BERT-MTB [216] learns relation representations based on BERT by performing the matching-the-blanks task and incorporating designed objectives for relation extraction. Curriculum-RE [160] utilizes curriculum learning to improve relation extraction models by gradually increasing the difficulty of the data during training. RECENT [217] introduces SpanBERT and exploits entity type restriction to reduce the noisy candidate relation types. Jiewen [218] extends RECENT by combining both the entity information and the label information into sentence-level embeddings, which enables the embedding to be entity-label aware.

Document-level RE (DocRE) aims to extract relations between entities across multiple sentences within a document. Hong et al. [219] propose a strong baseline for DocRE by replacing the BiLSTM backbone with LLMs. HIN [220] use LLM to encode and aggregate entity representation at different levels, including entity, sentence, and document levels. GLRE [221] is a global-to-local network, which uses LLM to encode the document information in terms of entity global and local representations as well as context relation representations. SIRE [222] uses two LLM-based encoders to extract intra-sentence and inter-sentence relations. LSR [223] and GAIN [224] propose graph-based approaches which induce graph structures on top of LLM to better extract relations. DocuNet [225] formulates DocRE as a semantic segmentation task and introduces a U-Net [226] on the LLM encoder to capture local and global dependencies between entities. ATLOP [227] focuses on the multi-label problems in DocRE, which could be handled with two techniques, i.e., adaptive thresholding for classifier and localized context pooling for LLM. DREEAM [161] further extends and improves ATLOP by incorporating evidence information.

End-to-End KG Construction. Currently, researchers are exploring the use of LLMs for end-to-end KG construction. Kumar et al. [95] propose a unified approach to build KGs from raw text, which contains two LLMs powered components. They first finetune a LLM on named entity recognition tasks to make it capable of recognizing entities in raw text. Then, they propose another "2-model BERT" for solving the relation extraction task, which contains two BERT-based classifiers. The first classifier learns the relation class whereas the second binary classifier learns the direction of the relations between the two entities. The predicted triples and relations are then used to construct the KG. Guo et al. [162] propose an end-to-end knowledge extraction model based on BERT, which can be applied to construct KGs from Classical Chinese text. Grapher [41] presents a novel end-to-end multi-stage system. It first utilizes LLMs to generate KG entities, followed by a simple relation construction head, enabling efficient KG construction from the textual description. PiVE [163] proposes a prompting with an iterative verification framework that utilizes a smaller LLM like T5 to correct the errors in KGs generated by a larger LLM (e.g., ChatGPT). To further explore advanced LLMs, AutoKG design several prompts for different KG construction tasks (e.g., entity typing, entity linking, and relation extraction). Then, it adopts the prompt to perform KG construction using ChatGPT and GPT-4.

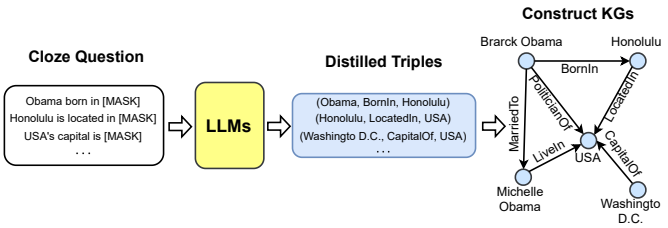

Fig. 20. The general framework of distilling KGs from LLMs.

5.3.4 Distilling Knowledge Graphs from LLMs

LLMs have been shown to implicitly encode massive knowledge [14]. As shown in Fig. 20, some research aims to distill knowledge from LLMs to construct KGs. COMET [164] proposes a commonsense transformer model that constructs commonsense KGs by using existing tuples as a seed set of knowledge on which to train. Using this seed set, a LLM learns to adapt its learned representations to knowledge generation, and produces novel tuples that are high quality. Experimental results reveal that implicit knowledge from LLMs is transferred to generate explicit knowledge in commonsense KGs. BertNet [165] proposes a novel framework for automatic KG construction empowered by LLMs. It requires only the minimal definition of relations as inputs and automatically generates diverse prompts, and performs an efficient knowledge search within a given LLM for consistent outputs. The constructed KGs show competitive quality, diversity, and novelty with a richer set of new and complex relations, which cannot be extracted by previous methods. West et al. [166] propose a symbolic knowledge distillation framework that distills symbolic knowledge from LLMs. They first finetune a small student LLM by distilling commonsense facts from a large LLM like GPT-3. Then, the student LLM is utilized to generate commonsense KGs.

5.4 LLM-augmented KG-to-text Generation

The goal of Knowledge-graph-to-text (KG-to-text) generation is to generate high-quality texts that accurately and consistently describe the input knowledge graph information [228]. KG-to-text generation connects knowledge graphs and texts, significantly improving the applicability of KG in more realistic NLG scenarios, including story-telling [229] and knowledge-grounded dialogue [230]. However, it is challenging and costly to collect large amounts of graph-text parallel data, resulting in insufficient training and poor generation quality. Thus, many research efforts resort to either: *1) leverage knowledge from LLMs* or *2) construct large-scale weakly-supervised KG-text corpus* to solve this issue.

5.4.1 Leveraging Knowledge from LLMs

As pioneering research efforts in using LLMs for KG-to-Text generation, Ribeiro et al. [167] and Kale and Rastogi [231] directly fine-tune various LLMs, including BART and T5, with the goal of transferring LLMs knowledge for this

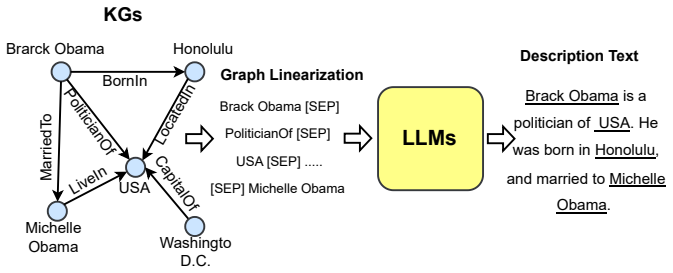

Fig. 21. The general framework of KG-to-text generation.

task. As shown in Fig. 21, both works simply represent the input graph as a linear traversal and find that such a naive approach successfully outperforms many existing state-of-the-art KG-to-text generation systems. Interestingly, Ribeiro et al. [167] also find that continue pre-training could further improve model performance. However, these methods are unable to *explicitly* incorporate rich graph semantics in KGs. To enhance LLMs with KG structure information, JointGT [42] proposes to inject KG structure-preserving representations into the Seq2Seq large language models. Given input sub-KGs and corresponding text, JointGT first represents the KG entities and their relations as a sequence of tokens, then concatenate them with the textual tokens which are fed into LLM. After the standard self-attention module, JointGT then uses a pooling layer to obtain the contextual semantic representations of knowledge entities and relations. Finally, these pooled KG representations are then aggregated in another structure-aware self-attention layer. JointGT also deploys additional pre-training objectives, including KG and text reconstruction tasks given masked inputs, to improve the alignment between text and graph information. Li et al. [168] focus on the few-shot scenario. It first employs a novel breadth-first search (BFS) strategy to better traverse the input KG structure and feed the enhanced linearized graph representations into LLMs for high-quality generated outputs, then aligns the GCN-based and LLM-based KG entity representation. Colas et al. [169] first transform the graph into its appropriate representation before linearizing the graph. Next, each KG node is encoded via a global attention mechanism, followed by a graph-aware attention module, ultimately being decoded into a sequence of tokens. Different from these works, KG-BART [37] keeps the structure of KGs and leverages the graph attention to aggregate the rich concept semantics in the sub-KG, which enhances the model generalization on unseen concept sets.

5.4.2 Constructing large weakly KG-text aligned Corpus

Although LLMs have achieved remarkable empirical success, their unsupervised pre-training objectives are not necessarily aligned well with the task of KG-to-text generation, motivating researchers to develop large-scale KG-text aligned corpus. Jin et al. [170] propose a 1.3M unsupervised KG-to-graph training data from Wikipedia. Specifically, they first detect the entities appearing in the text via hyperlinks and named entity detectors, and then only add text that shares a common set of entities with the corresponding knowledge graph, similar to the idea of distance supervision in the relation extraction task [232]. They also provide a 1,000+ human annotated KG-to-Text test data to verify the effectiveness of the pre-trained KG-to-Text models. Similarly, Chen et al. [171] also propose a KG-grounded text corpus collected from the English Wikidump. To ensure the connection between KG and text, they only extract sentences with at least two Wikipedia anchor links. Then, they use the entities from those links to query their surrounding neighbors in WikiData and calculate the lexical overlapping between these neighbors and the original sentences. Finally, only highly overlapped pairs are selected. The authors explore both graph-based and sequence-based encoders and identify their advantages in various different tasks and settings.

5.5 LLM-augmented KG Question Answering

Knowledge graph question answering (KGQA) aims to find answers to natural language questions based on the structured facts stored in knowledge graphs [233], [234]. The inevitable challenge in KGQA is to retrieve related facts and extend the reasoning advantage of KGs to QA. Therefore, recent studies adopt LLMs to bridge the gap between natural language questions and structured knowledge graphs [174], [175], [235]. The general framework of applying LLMs for KGQA is illustrated in Fig. 22, where LLMs can be used as 1) entity/relation extractors, and 2) answer reasoners.

5.5.1 LLMs as Entity/relation Extractors

Entity/relation extractors are designed to identify entities and relationships mentioned in natural language questions and retrieve related facts in KGs. Given the proficiency in language comprehension, LLMs can be effectively utilized for this purpose. Lukovnikov et al. [172] are the first to utilize LLMs as classifiers for relation prediction, resulting in a notable improvement in performance compared to shallow neural networks. Nan et al. [174] introduce two LLM-based KGQA frameworks that adopt LLMs to detect mentioned entities and relations. Then, they query the answer in KGs using the extracted entity-relation pairs. QA-GNN [131] uses LLMs to encode the question and candidate answer pairs, which are adopted to estimate the importance of relative KG entities. The entities are retrieved to form a subgraph, where an answer reasoning is conducted by a graph neural network. Luo et al. [173] use LLMs to calculate the similarities between relations and questions to retrieve related facts, formulated as

$$s(r, q) = \text{LLM}(r)^\top \text{LLM}(q), \tag{12}$$

where q denotes the question, r denotes the relation, and $\text{LLM}(\cdot)$ would generate representation for q and r, respectively. Furthermore, Zhang et al. [236] propose a LLM-based path retriever to retrieve question-related relations hop-by-hop and construct several paths. The probability of each path can be calculated as

$$P(p|q) = \prod_{t=1}^{|p|} s(r_t, q), \tag{13}$$

where p denotes the path, and r_t denotes the relation at the t-th hop of p. The retrieved relations and paths can be used

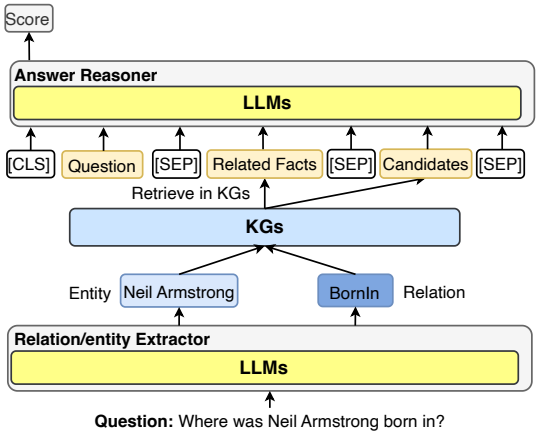

Fig. 22. The general framework of applying LLMs for knowledge graph question answering (KGQA).

as context knowledge to improve the performance of answer reasoners as

$$P(a|q) = \sum_{p \in \mathcal{P}} P(a|p)P(p|q), \tag{14}$$

where $\mathcal{P}$ denotes retrieved paths and a denotes the answer.

5.5.2 LLMs as Answer Reasoners

Answer reasoners are designed to reason over the retrieved facts and generate answers. LLMs can be used as answer reasoners to generate answers directly. For example, as shown in Fig. 3 22, DEKCOR [175] concatenates the retrieved facts with questions and candidate answers as

$$x = \texttt{[CLS]}\ q\ \texttt{[SEP]}\ \text{Related Facts}\ \texttt{[SEP]}\ a\ \texttt{[SEP]}, \tag{15}$$

where a denotes candidate answers. Then, it feeds them into LLMs to predict answer scores. After utilizing LLMs to generate the representation of x as QA context, DRLK [176] proposes a Dynamic Hierarchical Reasoner to capture the interactions between QA context and answers for answer prediction. Yan et al. [235] propose a LLM-based KGQA framework consisting of two stages: (1) retrieve related facts from KGs and (2) generate answers based on the retrieved facts. The first stage is similar to the entity/relation extractors. Given a candidate answer entity a, it extracts a series of paths $p_1, \ldots, p_n$ from KGs. But the second stage is a LLM-based answer reasoner. It first verbalizes the paths by using the entity names and relation names in KGs. Then, it concatenates the question q and all paths $p_1, \ldots, p_n$ to make an input sample as

$$x = \texttt{[CLS]}\ q\ \texttt{[SEP]}\ p_1\ \texttt{[SEP]}\ \cdots\ \texttt{[SEP]}\ p_n\ \texttt{[SEP]}. \tag{16}$$

These paths are regarded as the related facts for the candidate answer a. Finally, it uses LLMs to predict whether the hypothesis: "a is the answer of q" is supported by those facts, which is formulated as

$$e_{\texttt{[CLS]}} = \text{LLM}(x), \tag{17}$$

$$s = \sigma(\text{MLP}(e_{\texttt{[CLS]}})), \tag{18}$$

where it encodes x using a LLM and feeds representation corresponding to [CLS] token for binary classification, and $\sigma(\cdot)$ denotes the sigmoid function.

TABLE 4
Summary of methods that synergize KGs and LLMs.

Task	Method	Year
Synergized Knowledge representation	JointGT [42]	2021
	KEPLER [40]	2021
	DRAGON [44]	2022
	HKLM [238]	2023
Synergized Reasoning	LARK [45]	2023
	Siyuan et al. [46]	2023
	KSL [239]	2023
	StructGPT [237]	2023
	Think-on-graph [240]	2023

To better guide LLMs reason through KGs, OreoLM [177] proposes a Knowledge Interaction Layer (KIL) which is inserted amid LLM layers. KIL interacts with a KG reasoning module, where it discovers different reasoning paths, and then the reasoning module can reason over the paths to generate answers. GreaseLM [178] fuses the representations from LLMs and graph neural networks to effectively reason over KG facts and language context. UniKGQA [43] unifies the facts retrieval and reasoning into a unified framework. UniKGQA consists of two modules. The first module is a semantic matching module that uses a LLM to match questions with their corresponding relations semantically. The second module is a matching information propagation module, which propagates the matching information along directed edges on KGs for answer reasoning. Similarly, ReLMKG [179] performs joint reasoning on a large language model and the associated knowledge graph. The question and verbalized paths are encoded by the language model, and different layers of the language model produce outputs that guide a graph neural network to perform message passing. This process utilizes the explicit knowledge contained in the structured knowledge graph for reasoning purposes. StructGPT [237] adopts a customized interface to allow large language models (e.g., ChatGPT) directly reasoning on KGs to perform multi-step question answering.

6 Synergized LLMs + KGs

The synergy of LLMs and KGs has attracted increasing attention these years, which marries the merits of LLMs and KGs to mutually enhance performance in various downstream applications. For example, LLMs can be used to understand natural language, while KGs are treated as a knowledge base, which provides factual knowledge. The unification of LLMs and KGs could result in a powerful model for knowledge representation and reasoning.

In this section, we will discuss the state-of-the-art *Synergized LLMs + KGs* from two perspectives: *1) Synergized Knowledge Representation*, and *2)* Synergized Reasoning. Representative works are summarized in Table 4.

6.1 Synergized Knowledge Representation

Text corpus and knowledge graphs both contain enormous knowledge. However, the knowledge in text corpus is usually implicit and unstructured, while the knowledge in KGs is explicit and structured. Synergized Knowledge Representation aims to design a synergized model that can

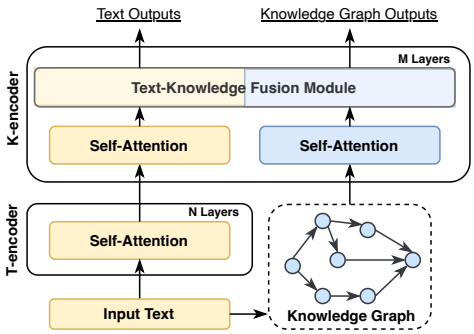

Fig. 23. Synergized knowledge representation by additional KG fusion modules.

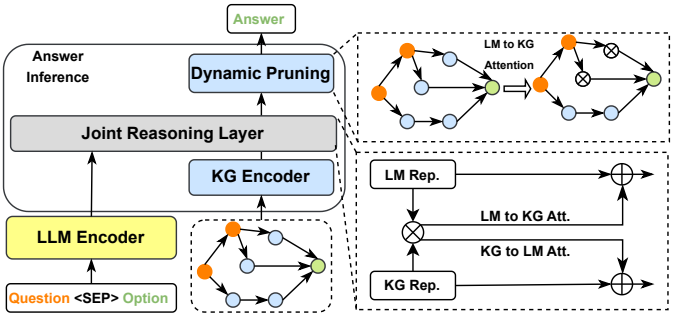

Fig. 24. The framework of LLM-KG Fusion Reasoning.

effectively represent knowledge from both LLMs and KGs. The synergized model can provide a better understanding of the knowledge from both sources, making it valuable for many downstream tasks.

To jointly represent the knowledge, researchers propose the synergized models by introducing additional KG fusion modules, which are jointly trained with LLMs. As shown in Fig. 23, ERNIE [35] proposes a textual-knowledge dual encoder architecture where a *T-encoder* first encodes the input sentences, then a *K-encoder* processes knowledge graphs which are fused them with the textual representation from the *T-encoder*. BERT-MK [241] employs a similar dual-encoder architecture but it introduces additional information of neighboring entities in the knowledge encoder component during the pre-training of LLMs. However, some of the neighboring entities in KGs may not be relevant to the input text, resulting in extra redundancy and noise. Coke-BERT [242] focuses on this issue and proposes a GNN-based module to filter out irrelevant KG entities using the input text. JAKET [243] proposes to fuse the entity information in the middle of the large language model.

KEPLER [40] presents a unified model for knowledge embedding and pre-trained language representation. In KEPLER, they encode textual entity descriptions with a LLM as their embeddings, and then jointly optimize the knowledge embedding and language modeling objectives. JointGT [42] proposes a graph-text joint representation learning model, which proposes three pre-training tasks to align representations of graph and text. DRAGON [44] presents a self-supervised method to pre-train a joint language-knowledge foundation model from text and KG. It takes text segments and relevant KG subgraphs as input and bidirectionally fuses information from both modalities. Then, DRAGON utilizes two self-supervised reasoning tasks, i.e., masked language modeling and KG link prediction to optimize the model parameters. HKLM [238] introduces a unified LLM which incorporates KGs to learn representations of domain-specific knowledge.

6.2 Synergized Reasoning

To better utilize the knowledge from text corpus and knowledge graph reasoning, Synergized Reasoning aims to design a synergized model that can effectively conduct reasoning with both LLMs and KGs.

LLM-KG Fusion Reasoning. LLM-KG Fusion Reasoning leverages two separated LLM and KG encoders to process the text and relevant KG inputs [244]. These two encoders are equally important and jointly fusing the knowledge from two sources for reasoning. To improve the interaction between text and knowledge, KagNet [38] proposes to first encode the input KG, and then augment the input textual representation. In contrast, MHGRN [234] uses the final LLM outputs of the input text to guide the reasoning process on the KGs. Yet, both of them only design a single-direction interaction between the text and KGs. To tackle this issue, QA-GNN [131] proposes to use a GNN-based model to jointly reason over input context and KG information via message passing. Specifically, QA-GNN represents the input textual information as a special node via a pooling operation and connects this node with other entities in KG. However, the textual inputs are only pooled into a single dense vector, limiting the information fusion performance. JointLK [245] then proposes a framework with fine-grained interaction between any tokens in the textual inputs and any KG entities through LM-to-KG and KG-to-LM bi-directional attention mechanism. As shown in Fig. 24, pairwise dot-product scores are calculated over all textual tokens and KG entities, the bi-directional attentive scores are computed separately. In addition, at each jointLK layer, the KGs are also dynamically pruned based on the attention score to allow later layers to focus on more important sub-KG structures. Despite being effective, in JointLK, the fusion process between the input text and KG still uses the final LLM outputs as the input text representations. GreaseLM [178] designs deep and rich interaction between the input text tokens and KG entities at each layer of the LLMs. The architecture and fusion approach is mostly similar to ERNIE [35] discussed in Section 6.1, except that GreaseLM does not use the text-only *T-encoder* to handle input text.

LLMs as Agents Reasoning. Instead using two encoders to fuse the knowledge, LLMs can also be treated as agents to interact with the KGs to conduct reasoning [246], as illustrated in Fig. 25. KD-CoT [247] iteratively retrieves facts from KGs and produces faithful reasoning traces, which guide LLMs to generate answers. KSL [239] teaches LLMs to search on KGs to retrieve relevant facts and then generate answers. StructGPT [237] designs several API interfaces to allow LLMs to access the structural data and perform reasoning by traversing on KGs. Think-on-graph [240] provides a flexible plug-and-play framework where LLM agents iteratively execute beam searches on KGs to discover the

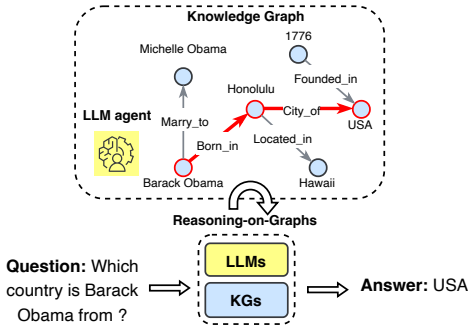

Fig. 25. Using LLMs as agents for reasoning on KGs.

reasoning paths and generate answers. To enhance the agent abilities, AgentTuning [248] presents several instruction-tuning datasets to guide LLM agents to perform reasoning on KGs.

Comparison and Discussion. LLM-KG Fusion Reasoning combines the LLM encoder and KG encoder to represent knowledge in a unified manner. It then employs a synergized reasoning module to jointly reason the results. This framework allows for different encoders and reasoning modules, which are trained end-to-end to effectively utilize the knowledge and reasoning capabilities of LLMs and KGs. However, these additional modules may introduce extra parameters and computational costs while lacking interpretability. LLMs as Agents for KG reasoning provides a flexible framework for reasoning on KGs without additional training cost, which can be generalized to different LLMs and KGs. Meanwhile, the reasoning process is interpretable, which can be used to explain the results. Nevertheless, defining the actions and policies for LLM agents is also challenging. The synergy of LLMs and KGs is still an ongoing research topic, with the potential to have more powerful frameworks in the future.

7 Future Directions and Milestones

In this section, we discuss the future directions and several milestones in the research area of unifying KGs and LLMs.

7.1 KGs for Hallucination Detection in LLMs

The hallucination problem in LLMs, which generates factually incorrect content, significantly hinders the reliability of LLMs. As discussed in Section 4, existing studies try to utilize KGs to obtain more reliable LLMs through pre-training or KG-enhanced inference. Despite the efforts, the issue of hallucination may continue to persist in the realm of LLMs for the foreseeable future. Consequently, in order to gain the public's trust and border applications, it is imperative to detect and assess instances of hallucination within LLMs and other forms of AI-generated content (AIGC). Existing methods strive to detect hallucination by training a neural classifier on a small set of documents [249], which are neither robust nor powerful to handle ever-growing LLMs. Recently, researchers try to use KGs as an external source to validate LLMs [250]. Further studies combine LLMs and KGs to achieve a generalized fact-checking model that can detect hallucinations across domains [251]. Therefore, it opens a new door to utilizing KGs for hallucination detection.

7.2 KGs for Editing Knowledge in LLMs

Although LLMs are capable of storing massive real-world knowledge, they cannot quickly update their internal knowledge updated as real-world situations change. There are some research efforts proposed for editing knowledge in LLMs [252] without re-training the whole LLMs. Yet, such solutions still suffer from poor performance or computational overhead [253]. Existing studies [254] also reveal that edit a single fact would cause a ripple effect for other related knowledge. Therefore, it is necessary to develop a more efficient and effective method to edit knowledge in LLMs. Recently, researchers try to leverage KGs to edit knowledge in LLMs efficiently.

7.3 KGs for Black-box LLMs Knowledge Injection

Although pre-training and knowledge editing could update LLMs to catch up with the latest knowledge, they still need to access the internal structures and parameters of LLMs. However, many state-of-the-art large LLMs (e.g., ChatGPT) only provide APIs for users and developers to access, making themselves black-box to the public. Consequently, it is impossible to follow conventional KG injection approaches described [38], [244] that change LLM structure by adding additional knowledge fusion modules. Converting various types of knowledge into different text prompts seems to be a feasible solution. However, it is unclear whether these prompts can generalize well to new LLMs. Moreover, the prompt-based approach is limited to the length of input tokens of LLMs. Therefore, how to enable effective knowledge injection for black-box LLMs is still an open question for us to explore [255], [256].

7.4 Multi-Modal LLMs for KGs

Current knowledge graphs typically rely on textual and graph structure to handle KG-related applications. However, real-world knowledge graphs are often constructed by data from diverse modalities [99], [257], [258]. Therefore, effectively leveraging representations from multiple modalities would be a significant challenge for future research in KGs [259]. One potential solution is to develop methods that can accurately encode and align entities across different modalities. Recently, with the development of multi-modal LLMs [98], [260], leveraging LLMs for modality alignment holds promise in this regard. But, bridging the gap between multi-modal LLMs and KG structure remains a crucial challenge in this field, demanding further investigation and advancements.

7.5 LLMs for Understanding KG Structure

Conventional LLMs trained on plain text data are not designed to understand structured data like knowledge graphs. Thus, LLMs might not fully grasp or understand the information conveyed by the KG structure. A straightforward way is to linearize the structured data into a sentence that LLMs can understand. However, the scale of the KGs

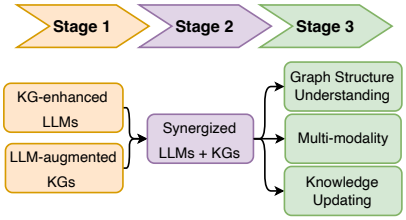

Fig. 26. The milestones of unifying KGs and LLMs.

makes it impossible to linearize the whole KGs as input. Moreover, the linearization process may lose some underlying information in KGs. Therefore, it is necessary to develop LLMs that can directly understand the KG structure and reason over it [237].

7.6 Synergized LLMs and KGs for Birectional Reasoning

KGs and LLMs are two complementary technologies that can synergize each other. However, the synergy of LLMs and KGs is less explored by existing researchers. A desired synergy of LLMs and KGs would involve leveraging the strengths of both technologies to overcome their individual limitations. LLMs, such as ChatGPT, excel in generating human-like text and understanding natural language, while KGs are structured databases that capture and represent knowledge in a structured manner. By combining their capabilities, we can create a powerful system that benefits from the contextual understanding of LLMs and the structured knowledge representation of KGs. To better unify LLMs and KGs, many advanced techniques need to be incorporated, such as multi-modal learning [261], graph neural network [262], and continuous learning [263]. Last, the synergy of LLMs and KGs can be applied to many real-world applications, such as search engines [100], recommender systems [10], [89], and drug discovery.

With a given application problem, we can apply a KG to perform a knowledge-driven search for potential goals and unseen data, and simultaneously start with LLMs to perform a data/text-driven inference to see what new data/goal items can be derived. When the knowledge-based search is combined with data/text-driven inference, they can mutually validate each other, resulting in efficient and effective solutions powered by dual-driving wheels. Therefore, we can anticipate increasing attention to unlock the potential of integrating KGs and LLMs for diverse downstream applications with both generative and reasoning capabilities in the near future.

8 Conclusion

Unifying large language models (LLMs) and knowledge graphs (KGs) is an active research direction that has attracted increasing attention from both academia and industry. In this article, we provide a thorough overview of the recent research in this field. We first introduce different manners that integrate KGs to enhance LLMs. Then, we introduce existing methods that apply LLMs for KGs and establish taxonomy based on varieties of KG tasks. Finally, we discuss the challenges and future directions in this field. We envision that there will be multiple stages (milestones) in the roadmap of unifying KGs and LLMs, as shown in Fig. 26. In particular, we will anticipate increasing research on three stages: *Stage 1*: KG-enhanced LLMs, LLM-augmented KGs, *Stage 2:* Synergized LLMs + KGs, and *Stage 3:* Graph Structure Understanding, Multi-modality, Knowledge Updating. We hope that this article will provide a guideline to advance future research.

Acknowledgments

This research was supported by the Australian Research Council (ARC) under grants FT210100097 and DP240101547 and the National Natural Science Foundation of China (NSFC) under grant 62120106008.

References

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
[2] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.
[3] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *The Journal of Machine Learning Research*, vol. 21, no. 1, pp. 5485–5551, 2020.
[4] D. Su, Y. Xu, G. I. Winata, P. Xu, H. Kim, Z. Liu, and P. Fung, "Generalizing question answering system with pre-trained language model fine-tuning," in *Proceedings of the 2nd Workshop on Machine Reading for Question Answering*, 2019, pp. 203–211.
[5] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in *ACL*, 2020, pp. 7871–7880.
[6] J. Li, T. Tang, W. X. Zhao, and J.-R. Wen, "Pretrained language models for text generation: A survey," *arXiv preprint arXiv:2105.10311*, 2021.
[7] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler *et al.*, "Emergent abilities of large language models," *Transactions on Machine Learning Research*.
[8] K. Malinka, M. Perešíni, A. Firc, O. Hujňák, and F. Januš, "On the educational impact of chatgpt: Is artificial intelligence ready to obtain a university degree?" *arXiv preprint arXiv:2303.11146*, 2023.
[9] Z. Li, C. Wang, Z. Liu, H. Wang, S. Wang, and C. Gao, "Cctest: Testing and repairing code completion systems," *ICSE*, 2023.
[10] J. Liu, C. Liu, R. Lv, K. Zhou, and Y. Zhang, "Is chatgpt a good recommender? a preliminary study," *arXiv preprint arXiv:2304.10149*, 2023.
[11] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, 2023.
[12] X. Qiu, T. Sun, Y. Xu, Y. Shao, N. Dai, and X. Huang, "Pre-trained models for natural language processing: A survey," *Science China Technological Sciences*, vol. 63, no. 10, pp. 1872–1897, 2020.
[13] J. Yang, H. Jin, R. Tang, X. Han, Q. Feng, H. Jiang, B. Yin, and X. Hu, "Harnessing the power of llms in practice: A survey on chatgpt and beyond," *arXiv preprint arXiv:2304.13712*, 2023.
[14] F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, and A. Miller, "Language models as knowledge bases?" in *EMNLP-IJCNLP*, 2019, pp. 2463–2473.
[15] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
[16] H. Zhang, H. Song, S. Li, M. Zhou, and D. Song, "A survey of controllable text generation using transformer-based pre-trained language models," *arXiv preprint arXiv:2201.05337*, 2022.